

Color Block Crush Solver

PlaySimple Hackathon • Sunil • Roll No: 22PD36

How it works

A level is a grid of coloured polyomino blocks that slide until they hit a wall, another block, or a matching-colour gate. Some blocks are locked to one axis; some stay ice-locked until N other blocks have exited; releasing one block can free another mid-move. `solve.py` reads a level and prints the moves, and every solution is replayed through the real simulator before it is printed, so a bug in the search can never slip a wrong answer through.

A state is one anchor cell per block plus a bitmask of who has exited. Identical blocks (four **DB** tiles in test4, eight **Y** tiles in test5) are grouped and sorted before hashing, which collapses 4! and 8! worth of duplicate states into a single canonical key. Moves are generated one slide-step at a time, not just the farthest stop, because at least one fixture only solves with a partial slide. Exits cascade: releasing one block re-checks every block waiting on a clear path, in a loop, until nothing more releases in that same transition – Test 5 needs exactly this to avoid a false **UNSOLVABLE**.

Heuristics

Two heuristics are kept separate on purpose. The admissible one is a backward BFS per block, from its own exit cells over a walls-only board, summed across every block still pending; it never overstates true cost, which is what makes **complete**'s shortest path and its **UNSOLVABLE** verdict trustworthy. The greedy one counts how many blocks sit in a pending block's straight-line path to its nearest exit, one level deep, plus a penalty for blocks still ice-locked – it is not admissible, but it is what turns Test 4's quarter-million-state search into a few thousand.

Search

All three solvers share one engine, weighted A* that reopens states instead of keeping a closed set (`search/astar.py`), $f(n) = g(n) + w \cdot h_{\text{adm}}(n) + w_g \cdot h_{\text{guide}}(n)$. A plain, uninformed BFS baseline also exists (`search/bfs.py`) purely as a control: it solves test1 instantly and times out on everything else, which is the actual evidence a heuristic is needed at all.

Solver	Algorithm	w	w_g	Optimal?
complete	weighted A*, one pass	1.0	0	Yes, proven
fast	weighted A*, one pass	3.0	3.0	No
auto	weighted A*, 3→1 schedule (ARA*-style)	3→1	3→0	Yes, proven

auto (the default) runs one search through a shrinking weight schedule instead of restarting from scratch each time. Every incumbent found mid-schedule tightens a bound used to prune the queue, and if the queue empties with an incumbent on hand, that incumbent is certified optimal, because pruning only ever uses the admissible term. **auto** is a bonus on top of the two solvers asked for – it costs no extra implementation since it is **complete** and **fast** sharing one run instead of two separate ones.

Results

Level	complete			fast			auto	
	Moves	Expanded	Time (s)	Moves	Expanded	Time (s)	Moves	Time (s)
test1	2	2	0.00	2	2	0.00	2	0.00
test2	10	10	0.00	11	11	0.01	10	0.01
test3	24	64,301	4.02	32	327	0.11	24	5.42
test4	49	259,396	16.61	58	5,807	3.48	49	20.05
test5	43	16,312	0.62	53	11,439	2.61	43	5.02

fast trades roughly nine to ten moves for 6-30x fewer expanded states on the two busy levels. **auto** makes no such trade – it matches **complete**'s move count on all five levels – and pays for it in wall time where **complete**

was already fast ($0.62\text{s} \rightarrow 5.02\text{s}$ on test5). All three run on the same move generator and exit simulator, so the only thing changing between them is how greedy the search is allowed to be; the trade here is moves against time, never moves against correctness.